

ROADMAP DE EVOLUÇÃO FULL-STACK

FabuLoja — Catálogo Digital + Leads

WhatsApp

Gerado em: 10/08/2026, 16:05:06

FABULOJA — ROADMAP DE EVOLUÇÃO FULL-STACK

Catálogo Full-Stack + Geração de Leads via WhatsApp

Versão estratégica • 2026

1. VISÃO DO PRODUTO

O FabuLoja não precisa se transformar em um e-commerce tradicional. O produto atual já possui uma proposta clara: catálogo digital, apresentação do produto e geração de intenção de compra direcionada ao WhatsApp. A evolução mantém esse fluxo e adiciona backend, persistência, administração e analytics.

Objetivo: Transformar o FabuLoja em uma plataforma full-stack de catálogo digital e geração de leads via WhatsApp, usando o próprio produto como laboratório para aprofundamento em backend, banco de dados, segurança, testes, Docker e deploy.

2. FLUXO-ALVO

Cliente !' Catálogo !' Produto !' "Quero comprar" !' API registra a intenção !' geração da mensagem !' WhatsApp

3. FASE 0 — BASE ATUAL

Objetivo: Preservar e estabilizar o produto existente

- Catálogo de produtos
- Categorias
- Detalhes do produto
- Responsividade
- Integração com WhatsApp
- Deploy atual

Marco da fase: o fluxo descoberta !' produto !' WhatsApp continua funcionando.

4. FASE 1 — FOUNDATION FRONT-END

Objetivo: Preparar o código para crescer

- Migrar componentes críticos para TypeScript
- Organizar tipos de Product, Category e Store
- Separar componentes e responsabilidades
- Padronizar ESLint + Prettier
- Revisar IDs/keys e dados duplicados

- Adicionar testes básicos de componentes

Marco da fase: Front-End tipado, organizado e preparado para consumir uma API real.

5. FASE 2 — BACKEND FOUNDATION

Objetivo: Construir a primeira API própria

- Node.js + TypeScript
- Fastify
- REST API
- Zod para validação
- Controller !' Service !' Data Access
- Tratamento centralizado de erros
- Variáveis de ambiente
- Status codes e contratos HTTP

Marco da fase: React deixa de depender de catálogo hardcoded e passa a consumir a API.

6. FASE 3 — POSTGRESQL + MODELAGEM

Objetivo: Aprender persistência e modelagem relacional

- PostgreSQL
- Prisma
- Migrations
- Relacionamentos
- Foreign keys e constraints
- Índices
- Paginação
- Consultas SQL fundamentais

Marco da fase: Store !' Categories !' Products + Store Settings.

7. FASE 4 — AUTENTICAÇÃO E AUTORIZAÇÃO

Objetivo: Criar uma área administrativa segura

- Cadastro/login administrativo
- Password hashing
- Sessão/token
- Cookies HTTP-only
- Proteção de rotas
- ADMIN / CUSTOMER
- RBAC
- Respostas 401 e 403

Marco da fase: somente usuários autorizados conseguem administrar o catálogo.

8. FASE 5 — PAINEL ADMINISTRATIVO

Objetivo: Transformar o FabuLoja em uma plataforma gerenciável

- Dashboard
- CRUD de produtos
- CRUD de categorias
- Ativar/desativar produtos
- Preço e descrição
- Gestão de imagens

- Configuração do WhatsApp
- Configurações da loja

Marco da fase: o catálogo deixa de depender de alterações manuais no código.

9. FASE 6 — WHATSAPP + LEADS

Objetivo: Transformar o clique de compra em evento de negócio

- Registrar `product_view`
- Registrar `buy_click`
- Registrar `whatsapp_redirect`
- Associar evento ao produto
- Timestamp e contexto da sessão
- Gerar mensagem dinâmica com nome, preço e URL
- Manter o WhatsApp como destino final

Marco da fase: Product View !' Buy Click !' Lead !' WhatsApp Redirect.

10. FASE 7 — ANALYTICS

Objetivo: Medir intenção de compra e desempenho do catálogo

- Visualizações por produto
- Cliques em "Quero comprar"
- Leads gerados
- Produtos mais acessados
- Produtos com maior intenção
- Conversão de visualização !' clique
- Filtros por período

Marco da fase: o FabuLoja passa a responder quais produtos geram mais interesse comercial.

11. FASE 8 — QUALIDADE E SEGURANÇA

Objetivo: Sair do "funciona" para engenharia de aplicação

- Validação de payloads
- Tratamento de erros
- CORS
- Rate limiting
- Headers de segurança
- Secrets e environment variables
- Logs
- Revisão de dependências
- Regras de autorização

Marco da fase: API preparada para uso real e manutenção.

12. FASE 9 — TESTES

Objetivo: Garantir comportamento do sistema

- Unit tests para regras de negócio
- Integration tests API + banco
- Testes de autenticação
- Testes de autorização
- Testes de produtos/categorias
- Teste do fluxo de lead

- E2E do fluxo principal

Marco da fase: login !' admin !' criar produto !' loja !' clique !' lead !' WhatsApp.

13. FASE 10 — DOCKER + CI/CD

Objetivo: Profissionalizar o ciclo de entrega

- Dockerfile da API
- Docker Compose
- PostgreSQL em container
- CI no GitHub Actions
- Lint
- Typecheck
- Testes
- Build
- Deploy automatizado

Marco da fase: Push/PR !' Lint !' Typecheck !' Tests !' Build !' Deploy.

14. FASE 11 — PRODUÇÃO + OBSERVABILIDADE

Objetivo: Colocar o sistema completo em produção

- Frontend publicado
- API publicada
- PostgreSQL em produção
- HTTPS
- Environment variables
- Logs de produção
- Monitoramento básico
- Backup/restore do banco
- Documentação da API

Marco da fase: FabuLoja operando como aplicação full-stack real.

15. ARQUITETURA ALVO

A arquitetura deve continuar simples o suficiente para ser compreendida e mantida. O objetivo é aprender separação de responsabilidades, não criar microservices.

```
STORE FRONTEND / ADMIN PANEL
!"
REST API
!"
Controller !' Service !' Data Access
!"
PostgreSQL
```

16. STACK RECOMENDADA

- Frontend: React + TypeScript + Vite + Tailwind
- Backend: Node.js + TypeScript + Fastify
- Validation: Zod
- Database: PostgreSQL
- ORM: Prisma
- Testing: Vitest + React Testing Library + Playwright
- Infra: Docker + GitHub Actions

17. FORA DO ESCOPO INICIAL

Para evitar over-engineering e manter o projeto alinhado ao modelo real do FabuLoja, não são requisitos iniciais:

- Checkout tradicional
- Gateway de pagamento
- Frete
- Wishlist
- Login social
- Microservices
- Kubernetes
- Filas complexas

Esses recursos só entram se houver necessidade de produto ou como estudo posterior.

18. RESULTADO PROFISSIONAL ESPERADO

Ao concluir o roadmap, o FabuLoja deverá demonstrar não apenas competência de Front-End, mas capacidade de construir uma aplicação web completa: API REST, banco relacional, autenticação, autorização, regras de negócio, analytics, testes, segurança, containerização, CI/CD e deploy.

19. CRITÉRIO DE CONSOLIDAÇÃO

O objetivo não é apenas terminar funcionalidades. A consolidação acontece quando você consegue explicar e defender as decisões técnicas: arquitetura, modelagem do banco, proteção de rotas, tratamento de erros, testes, consistência dos dados e deploy.

FabuLoja !' de catálogo Front-End para produto Full-Stack real.